```
orders.size
orders.find(...)
orders.where(...)
orders.exists?(...)
orders.build(attributes = {}, ...)
orders.create(attributes = {})
orders.create!(attributes = {})
```

4.3.1.1 collection(force_reload = false)

The `collection` method returns an array of all of the associated objects. If there are no associated objects, it returns an empty array.

```
@orders = @customer.orders
```

4.3.1.2 collection<<(object, ...)

The `collection<<` method adds one or more objects to the collection by setting their foreign keys to the primary key of the calling model.

```
@customer.orders << @order1
```

4.3.1.3 collection.delete(object, ...)

The `collection.delete` method removes one or more objects from the collection by setting their foreign keys to `NULL`.

```
@customer.orders.delete(@order1)
```

Additionally, objects will be destroyed if they're associated with `dependent: :destroy`, and deleted if they're associated with `dependent: :delete_all`.

4.3.1.4 collection.destroy(object, ...)

The `collection.destroy` method removes one or more objects from the collection by running `destroy` on each object.

```
@customer.orders.destroy(@order1)
```

Objects will *always* be removed from the database, ignoring the `:dependent` option.

4.3.1.5 collection=(objects)

The `collection=` method makes the collection contain only the supplied objects, by adding and deleting as appropriate.

4.3.1.6 collection_singular_ids

The `collection_singular_ids` method returns an array of the ids of the objects in the collection.

```
@order_ids = @customer.order_ids
```

4.3.1.7 collection_singular_ids=(ids)

The `collection_singular_ids=` method makes the collection contain only the objects identified by the supplied primary key values, by adding and deleting as appropriate.

4.3.1.8 collection.clear

The `collection.clear` method removes every object from the collection. This destroys the associated objects if they are associated with `dependent: :destroy`, deletes them directly from the database if `dependent: :delete_all`, and otherwise sets their foreign keys to `NULL`.

4.3.1.9 collection.empty?

The `collection.empty?` method returns `true` if the collection does not contain any associated objects.

```
<% if @customer.orders.empty? %>
  No Orders Found
<% end %>
```

4.3.1.10 collection.size

The `collection.size` method returns the number of objects in the collection.

```
@order_count = @customer.orders.size
```

4.3.1.11 collection.find(...)

The `collection.find` method finds objects within the collection. It uses the same syntax and options as `ActiveRecord::Base.find`.

```
@open_orders = @customer.orders.find(1)
```

4.3.1.12 collection.where(...)